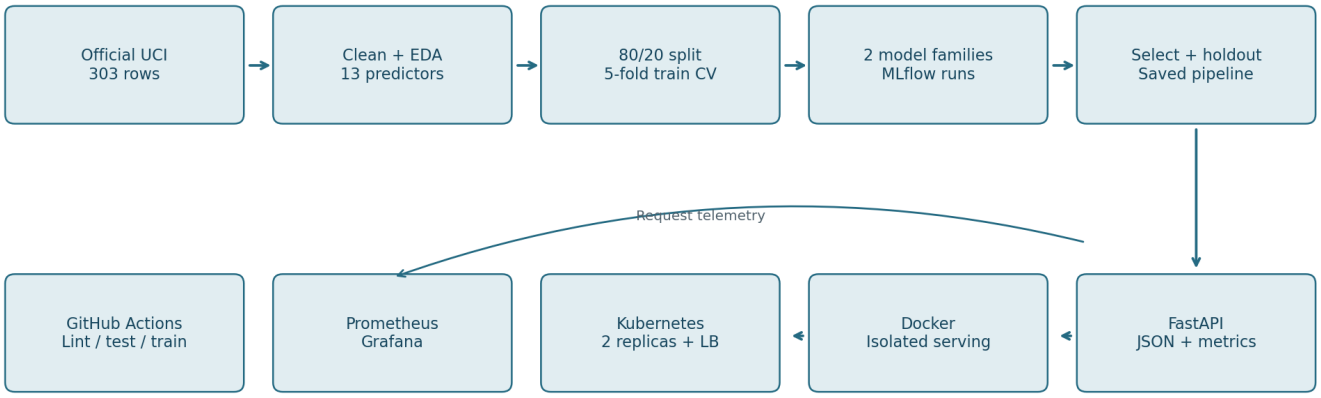
01 Project overview

Purpose and submission identity

A reproducible classifier for disease presence in the historical UCI Cleveland cohort. The implementation follows the full assignment lifecycle: acquisition, EDA, model comparison, tracking, packaging, testing, serving, deployment configuration and request monitoring.

Student	BHAMMAR MITUL HARSHADRAI
Student ID	2025ae05118
Repository	https://github.com/mitulbhammar-bits/heart-disease-mlops

Architecture and artifact flow



Logical architecture: data, training and MLflow, packaged pipeline, FastAPI container, Kubernetes Service and Prometheus/Grafana monitoring.

Measured outcome and completion boundary

The selected estimator is **logistic regression**, selected by training-only mean CV ROC-AUC (0.902). It uses 242 training and 61 holdout patients. Python/API, Docker, Minikube, GitHub Actions and Grafana runs are recorded in verification_status.json with screenshots in the screenshots folder.

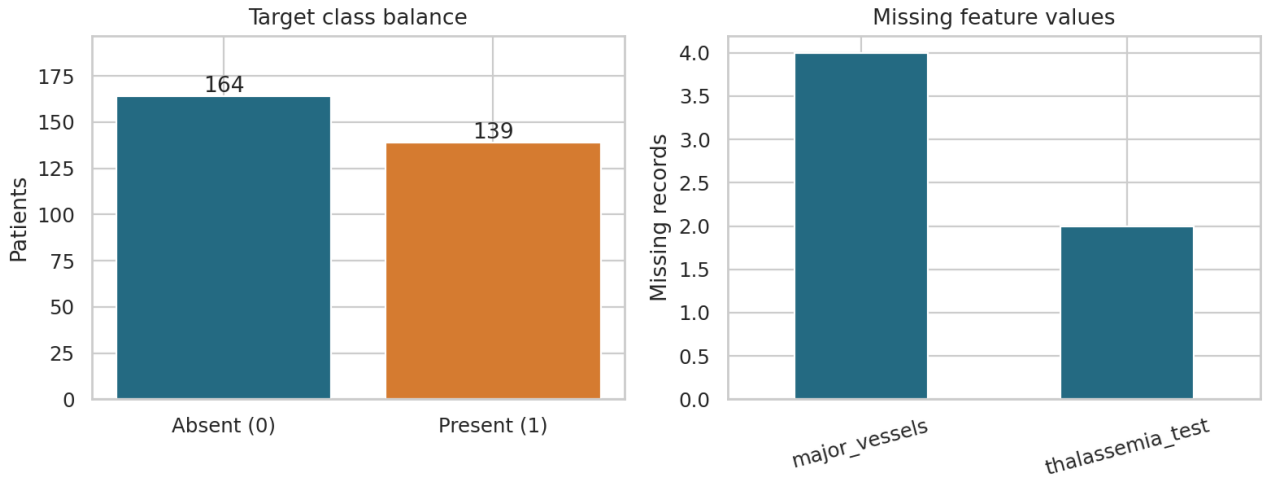
Deliverables: GitHub repository (link above), CI workflow, Dockerfile, Kubernetes manifests, monitoring configuration, screenshots, this report and a recorded pipeline walkthrough.

02 Data acquisition and cleaning

The download module retrieves the official UCI ZIP and extracts processed.cleveland.data. It retains the source file, records SHA-256 checksums and writes human-readable column names to data/processed/heart_disease.csv. Cached raw data is checksum-verified on subsequent runs.

Property	Verified value
Rows	303
Predictors	13
Target counts	164 absent; 139 present
Missing values	major_vessels: 4; thalassemia_test: 2
Exact duplicate rows	0

The standard processed UCI dataset has 13 predictors and one target, rather than 14+ predictors. Original diagnosis codes 1 through 4 map to disease present (1); code 0 maps to absent (0). The original '?' markers become missing values. No target values are missing.



The cohort is moderately balanced; six cells require fold-local imputation.

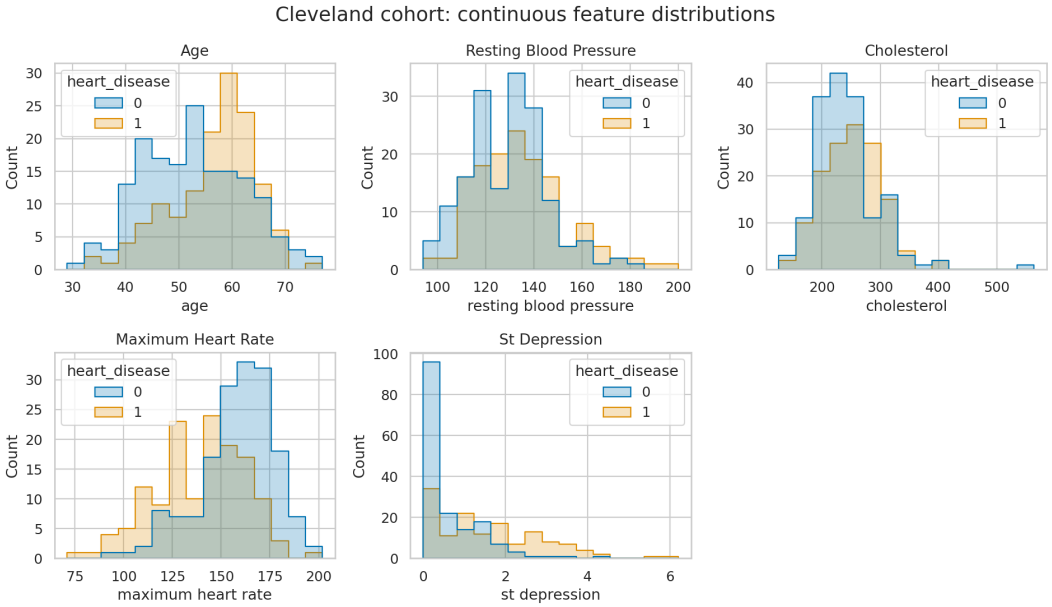
Why the cleaned CSV still contains blanks

Cleaning normalizes names and values but does not learn replacement statistics globally. Median and most-frequent imputers are fitted inside the estimator pipeline so validation and holdout rows never influence imputation. No rows are dropped merely because a feature is missing.

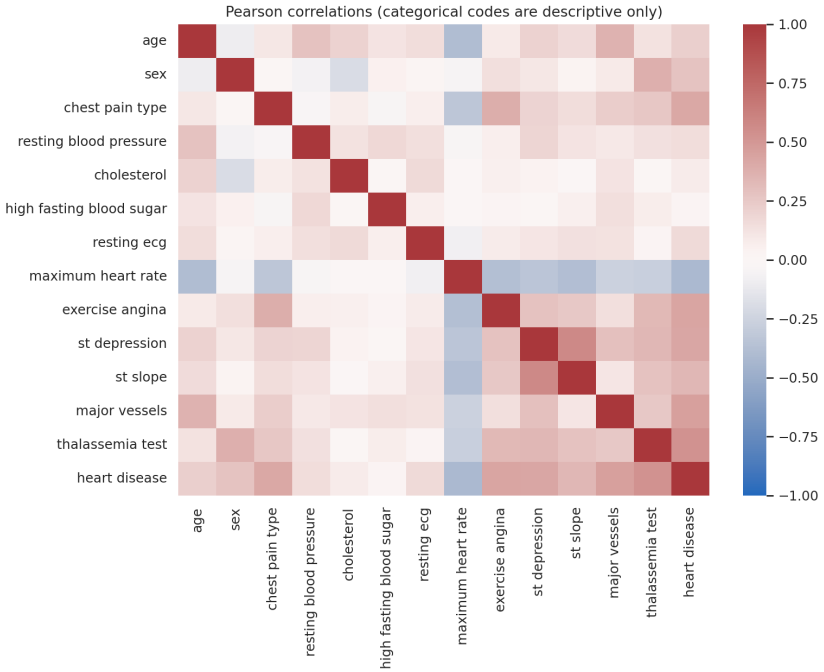
Source and attribution: Janosi, Steinbrunn, Pfisterer and Detrano (1989), Heart Disease, UCI Machine Learning Repository; DOI 10.24432/C52P4X. Included data is licensed CC BY 4.0.

03 Exploratory data analysis

The figures inspect continuous feature distributions by class, missingness and pairwise correlations. These descriptive views help explain the cohort; they do not determine hyperparameters or fit preprocessing. The feature set is predefined from the standard UCI variables.



Histograms show overlap between classes; no single continuous variable perfectly separates them.



Pearson heatmap. Correlations between arbitrary categorical codes are descriptive, not causal.

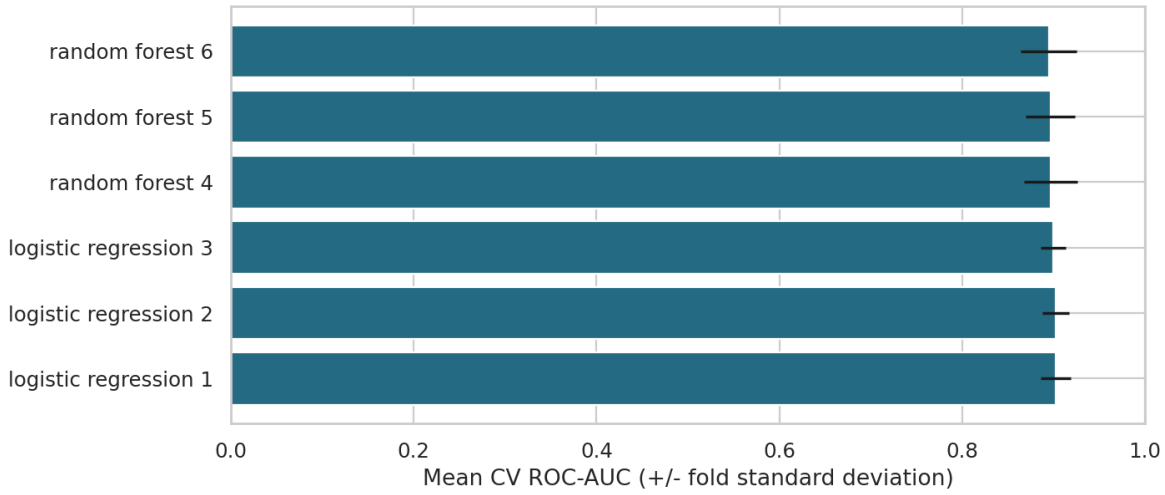
The small historical sample limits subgroup analysis and external generalization. Values and coding are documented in data/data_dictionary.md. This target describes observed disease presence and must not be interpreted as a validated future-risk estimate.

04 Features and model selection

A stratified 80/20 split (seed 42) reserves the holdout. Five stratified training folds use the same seed and partitions for every candidate. Continuous features receive median imputation and StandardScaler; categorical features receive most-frequent imputation and one-hot encoding. Unknown categories are ignored by the transformer, while the public API rejects invalid UCI codes.

Logistic Regression provides a simple regularized baseline; C = 0.1, 1 and 10 explores regularization strength. Random Forest captures interactions; maximum depths 3, 6 and unrestricted are explored with 200 trees and minimum leaf size 2. The intentionally small grid keeps experiments auditable.

Candidate	Settings	CV ROC-AUC	CV recall
logistic regression	{C: 0.1}	0.902 +/- 0.017	0.774
logistic regression	{C: 1.0}	0.902 +/- 0.014	0.783
logistic regression	{C: 10.0}	0.899 +/- 0.014	0.802
random forest	{max_depth: 3, min_samples_leaf: 2}	0.897 +/- 0.029	0.764
random forest	{max_depth: 6, min_samples_leaf: 2}	0.897 +/- 0.027	0.764
random forest	{max_depth: None, min_samples_leaf: 2}	0.895 +/- 0.031	0.783



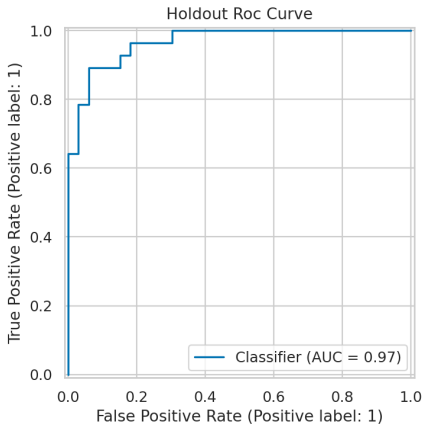
Error bars represent fold standard deviation, not a confidence interval.

Selection uses the highest mean CV ROC-AUC only. Equal scores retain the first candidate. The full candidate table also reports accuracy, precision and recall with fold variability. The holdout is evaluated only after selecting the winner; it is not used to tune the threshold.

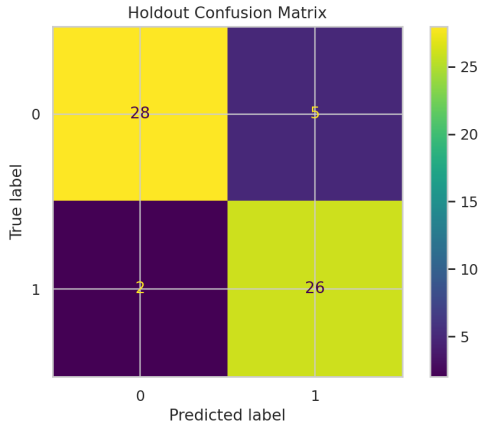
05 Held-out model evaluation

The winner is logistic regression with parameters {'C': 1.0}. The serving artifact remains fitted on the training partition, so the reported holdout metrics directly describe the packaged model. No refit on the holdout is performed. Binary predictions use threshold 0.5.

Metric	Untouched holdout result
Accuracy	0.8852
Precision	0.8387
Recall	0.9286
Roc Auc	0.9665



ROC curve: ranking performance across thresholds.



Confusion matrix at threshold 0.5: rows are actual, columns predicted.

Recall describes the proportion of positive cases detected; precision describes the positive predictive fraction in this cohort. Accuracy depends on class balance, while ROC-AUC measures ranking. A 61-patient test set yields uncertain estimates: these results are not clinical validation. Calibration, external validation and threshold design remain future work.

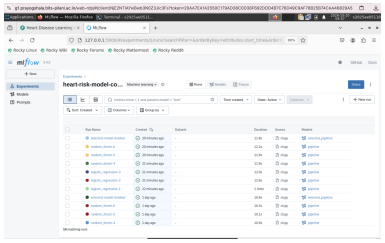
06 Experiment tracking

MLflow uses a local SQLite tracking store and an explicit artifact directory. Every one of the six candidate runs records its family, hyperparameters, seed, fold count and training size; mean and standard deviation of four CV metrics; all fold scores; a CV plot; source provenance; and a fitted full pipeline with an input example. A seventh run records the selected model's holdout evaluation.

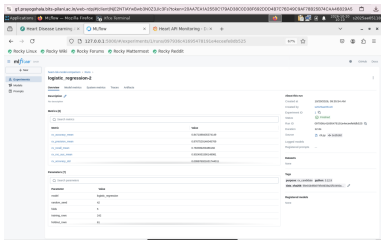
Tracked item	How to inspect it
Parameters and CV metrics	Compare candidate runs in the MLflow experiment
Fold variability	fold_metrics.json and plots/cross_validation.png
Data and environment identity	SHA-256 tag, source_manifest.json, Python tag
Reusable estimator	Logged sklearn pipeline and models/heart_risk_pipeline.joblib
Final evaluation	selected-model-holdout run, ROC and confusion matrix artifacts

Open the tracking interface

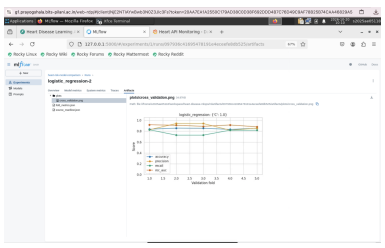
```
mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
```



MLflow UI: candidate runs and the selected-model holdout run.



One run: cross-validation metrics, parameters, tags and source commit.



The same run's artifacts: cross-validation plot, fold metrics and source manifest.

For portable reproduction, rerun training after moving the project: older SQLite run records may retain absolute artifact locations. The joblib model has no custom project transformer and works with the pinned serving dependencies. Tracked artifacts and explicit data hashes support auditing, but are not a managed production model registry or drift-detection service.

07 Reproducibility and API

Use Python 3.12 and install requirements-lock.txt in a fresh virtual environment. Direct dependencies are also pinned in requirements.txt. A SQLAlchemy 2.0 pin avoids an observed MLflow 3.4 tracking-store incompatibility. Deterministic split/fold seeds, data checksums, saved split indices and a full preprocessing pipeline make experiments reproducible.

The saved model also passed live HTTP checks in a fresh serving-only virtual environment, with only code and model copied and no training data or MLflow dependencies. This verifies Python environment isolation; it does not replace Docker execution proof.

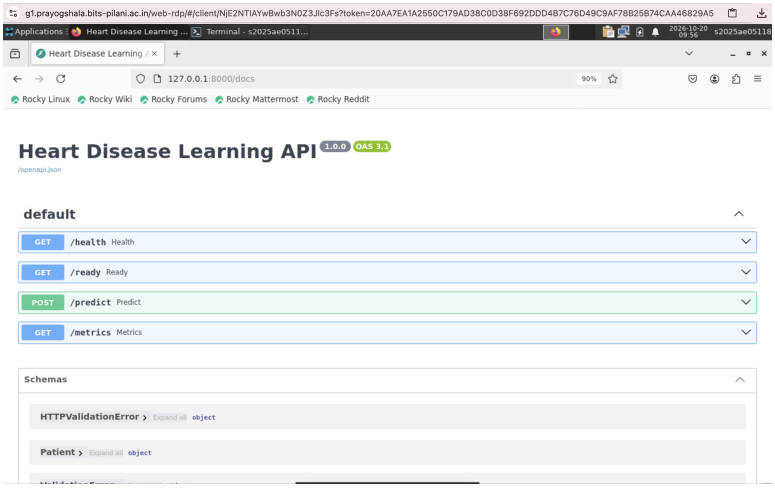
Run the core stages

```
python -m heart_risk.cli all
python -m heart_risk.cli predict
python -m uvicorn heart_risk.api:app --host 127.0.0.1 --port 8000
python scripts/verify_endpoint.py
```

Endpoint	Contract
POST /predict	One JSON patient; returns class, label, positive probability, confidence, threshold and version
GET /health; GET /ready	Liveness and startup-loaded model readiness
GET /metrics; GET /docs	Prometheus metrics and interactive OpenAPI documentation

Measured sample response: prediction = 0, disease probability = 0.1655, confidence = 0.8345. The real HTTP verification checked health/readiness/metrics and an invalid request returning 422.

Confidence is the probability of the returned class: p for class 1, 1-p for class 0. It is not a validated estimate of clinical certainty. The public schema rejects extra keys, non-finite numbers, missing required fields and out-of-range category codes. Optional UCI missing fields are transformed to NaN and imputed by the saved pipeline.



Local API run.

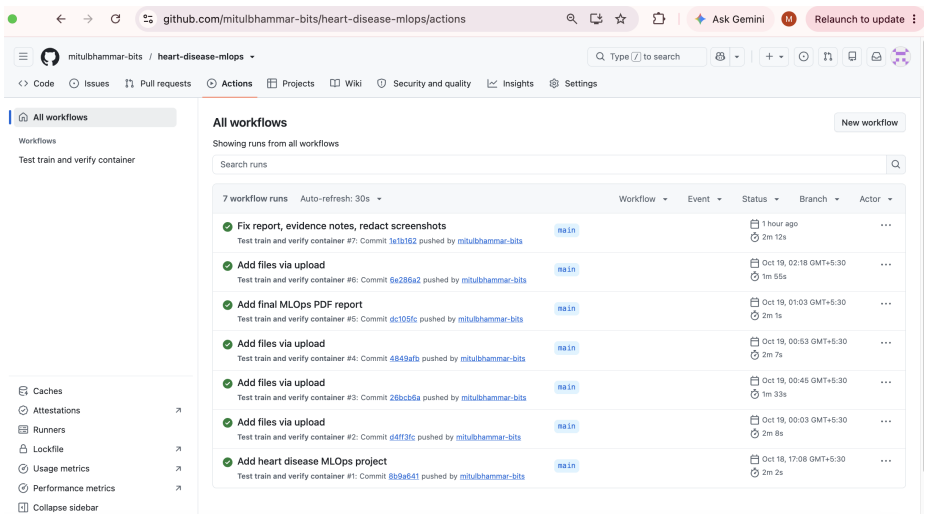
Only load model artifacts from trusted sources: joblib serialization is intended for this controlled project workflow. The same serialization prediction behavior is tested after reload.

08 Tests and CI workflow

Tests focus on behavior: source schema/checksum, all five diagnosis mappings, missing data, both model families, unseen categories, serialization, training-only imputer statistics, JSON validation, probability/confidence, health/readiness, metrics and missing-model startup failure.

Local verification: 26 passed; 0 failures; 0 errors. Lint: Passed ruff checks. Machine-readable results are saved in reports/evidence/test_results.xml.

Workflow stage	Failure and evidence behavior
Install and lint	Pinned environment; nonzero exit fails the job
Pytest	Behavior tests; JUnit result retained
EDA and model training	All reproducible stages; models and report data retained
Saved-model inference	Checks pipeline load and prediction
Docker smoke test	Fresh image/container; real endpoint verification
Artifact upload	Runs even on failure; logs, models, data provenance and MLflow artifacts



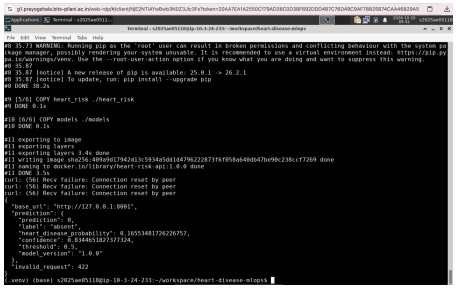
Successful GitHub Actions run of the CI workflow.

The supplied workflow validates a deployable artifact on push, pull request or manual dispatch. The local Kubernetes rollout is an explicit next stage because hosted runners do not control a student's Minikube cluster; the green hosted run above covers lint, tests, training, inference and the Docker smoke test. A public-cloud automatic rollout would require explicit credentials and a deployment environment.

09 Container and Kubernetes

The image uses Python 3.12 slim, only serving dependencies, and a non-root UID. It copies the trained pipeline, exposes port 8000 and checks readiness. verify_docker.sh builds a fresh image, starts an isolated container on host port 8001 and records API validation, build/runtime logs and inspection data.

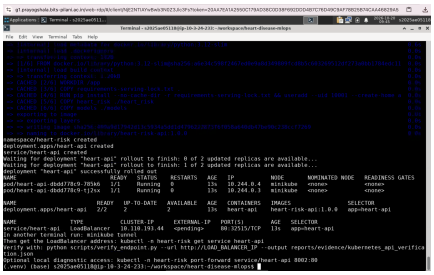
Container status: **Executed 2026-10-20 on lab host: image heart-risk-api:1.0.0 built, isolated container on port 8001 returned /predict 200 and invalid input 422 (screenshots/docker_verification.png)**.



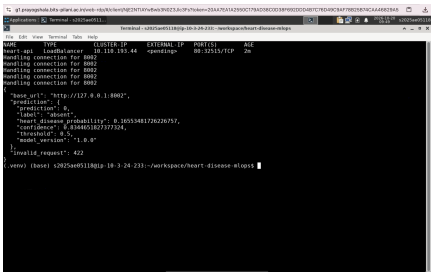
Docker image build and isolated container prediction.

Local Kubernetes deployment route

The manifests define a dedicated namespace, two replicas, startup/readiness/liveness probes, resource requests/limits and a LoadBalancer service. The Minikube script loads the local image and waits for rollout. The Service is of type LoadBalancer, but minikube tunnel on port 80 was not usable on the remote lab host, so its EXTERNAL-IP stayed pending. The endpoint was therefore verified with kubectl port-forward to the Service (127.0.0.1:8002); on a cloud cluster the same manifest receives an external IP.



Two ready replicas and the LoadBalancer Service.



/predict served by the Kubernetes Service through port-forward.

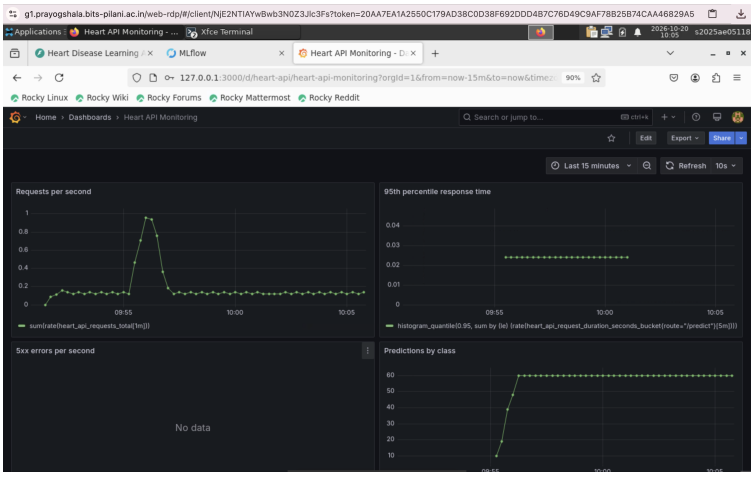
Access instructions/status: Local Minikube LoadBalancer Service, verified via kubectl port-forward: http://127.0.0.1:8002/predict (see reports/evidence/kubernetes_access_note.md)

For recovery, inspect pod logs and events; verify the image is loaded and the model exists. A failed readiness probe prevents traffic; kubectl rollout undo restores a previous deployment. Deployment used local Minikube, not a public cloud cluster.

10 Monitoring and handoff

Every API request produces a structured log with a generated request ID, method, route, status and elapsed milliseconds. Patient bodies are not logged. Prometheus counters record request/status totals and predicted classes; a histogram records latency. Low-cardinality route labels avoid unbounded metric series.

The Compose monitoring stack runs the API, Prometheus and provisioned Grafana dashboard. Dashboard panels show request rate, p95 prediction latency, server errors and class counts. The screenshot was taken after sending real prediction requests. This is service monitoring; ground-truth-driven model quality or drift alerts are not implemented.



Grafana dashboard with live API metrics.

Submission summary

All nine assignment areas have executed evidence: reproducible EDA and training, MLflow runs, packaged pipeline, tests and CI, Docker run, Kubernetes deployment and Grafana monitoring. The Kubernetes endpoint was reached by port-forward as explained in section 09.

Video: reports/pipeline_walkthrough.mp4

References and understanding

UCI Heart Disease: archive.ics.uci.edu/dataset/45/heart+disease; DOI 10.24432/C52P4X (CC BY 4.0). MLflow: mlflow.org/docs/latest/ml/tracking/. FastAPI: fastapi.tiangolo.com/deployment/docker/. scikit-learn: scikit-learn.org/stable/modules/compose.html. Kubernetes: kubernetes.io/docs/concepts/services-networking/service/.

Explain why imputation belongs inside CV, why selection excludes the holdout, what probability and confidence mean, how CI surfaces failures, and why liveness differs from readiness. Personal experimentation and a live demonstration support the independent-work requirement.